

Instructions:

You will work on these exercises during the practical sessions in the fourth week of the course.

Below is an indication of the exercises that we will cover (at least) in each session. We encourage you try them before the session.

- First session (Wednesday/Thursday): exercises 1 and 3.
- Second session (Friday): exercise 2.

Exercise 1: Multi-class Logistic Regression

In the binary case, logistic regression models posterior class probabilities with a sigmoid. For $K > 2$ classes, the posterior probabilities are modeled with the softmax function:

$$y_k = p(\mathcal{C}_k \mid \phi) = \frac{\exp(a_k)}{\sum_{i=1}^K \exp(a_i)},$$

where

$$a_k = \mathbf{w}_k^\top \phi, \quad \phi = \phi(\mathbf{x}) = (\phi_0(\mathbf{x}), \phi_1(\mathbf{x}), \dots, \phi_{M-1}(\mathbf{x}))^\top,$$

with $\phi_0(\mathbf{x}) = 1$. Note that y_k depends on all class weight vectors $\mathbf{w}_1, \dots, \mathbf{w}_K$ through the denominator of the softmax.

The training set consists of a design matrix Φ and a target matrix $\mathbf{T}$. Each row of $\mathbf{T}$ is one-hot encoded: the n -th row is $\mathbf{t}_n^\top = (t_{n1}, \dots, t_{nK})$, where $t_{nk} = 1$ if data point n belongs to class $\mathcal{C}_k$ and $t_{nk} = 0$ otherwise.

- (a) Derive $\frac{\partial y_k}{\partial \mathbf{w}_j}$. Use the indicator function or Kronecker Delta (δ_{ij}).

Solution:

We use the chain rule through the activations $a_1, \dots, a_K$:

$$\frac{\partial y_k}{\partial \mathbf{w}_j} = \sum_{p=1}^K \frac{\partial y_k}{\partial a_p} \frac{\partial a_p}{\partial \mathbf{w}_j}.$$

Since $a_p = \mathbf{w}_p^\top \phi$,

$$\frac{\partial a_p}{\partial \mathbf{w}_j} = \phi^\top \mathbb{I}[p = j].$$

To compute $\partial y_k / \partial a_p$, write $C = \sum_{i=1}^K \exp(a_i)$, so $y_k = \exp(a_k) / C$. Using the quotient rule,

$$\begin{aligned} \frac{\partial y_k}{\partial a_p} &= \frac{\delta_{kp} \exp(a_k) C - \exp(a_k) \exp(a_p)}{C^2} \\ &= \delta_{kp} y_k - y_k y_p \\ &= y_k (\delta_{kp} - y_p). \end{aligned}$$

Therefore only the term $p = j$ remains in the chain rule sum, and

$$\frac{\partial y_k}{\partial \mathbf{w}_j} = y_k (\delta_{kj} - y_j) \phi^\top.$$

- (b) Write down the likelihood $p(\mathbf{T} \mid \Phi, \mathbf{w}_1, \dots, \mathbf{w}_K)$ as a product over N and K . Use the entries of $\mathbf{T}$ as selectors of the correct class. Then write down the log-likelihood.

Solution:

For data point n , let $y_{nk} = y_k(\phi_n)$. Using independence over data points and the one-hot entries as selectors,

$$p(\mathbf{T} \mid \Phi, \mathbf{w}_1, \dots, \mathbf{w}_K) = \prod_{n=1}^N \prod_{k=1}^K y_{nk}^{t_{nk}}.$$

Taking logarithms gives

$$\ln p(\mathbf{T} \mid \Phi, \mathbf{w}_1, \dots, \mathbf{w}_K) = \sum_{n=1}^N \sum_{k=1}^K t_{nk} \ln y_{nk}.$$

- (c) Derive the gradient of the log-likelihood with respect to $\mathbf{w}_j$.

Solution:

Starting from the log-likelihood,

$$\ell = \sum_{n=1}^N \sum_{k=1}^K t_{nk} \ln y_{nk},$$

we differentiate with respect to $\mathbf{w}_j$:

$$\begin{aligned} \frac{\partial \ell}{\partial \mathbf{w}_j} &= \sum_{n=1}^N \sum_{k=1}^K t_{nk} \frac{\partial y_{nk}}{y_{nk}} \frac{\partial y_{nk}}{\partial \mathbf{w}_j} \\ &= \sum_{n=1}^N \sum_{k=1}^K t_{nk} (\delta_{kj} - y_{nj}) \phi_n^\top \\ &= \sum_{n=1}^N \left(t_{nj} - y_{nj} \sum_{k=1}^K t_{nk} \right) \phi_n^\top. \end{aligned}$$

Since each target vector is one-hot, $\sum_{k=1}^K t_{nk} = 1$. Hence

$$\frac{\partial \ell}{\partial \mathbf{w}_j} = \sum_{n=1}^N (t_{nj} - y_{nj}) \phi_n^\top.$$

Equivalently, as a column-vector gradient,

$$\nabla_{\mathbf{w}_j} \ell = \sum_{n=1}^N (t_{nj} - y_{nj}) \phi_n.$$

- (d) What objective function do we minimize that is equivalent to maximizing the log-likelihood?

Solution:

Maximizing the log-likelihood is equivalent to minimizing the negative log-likelihood, also

called the cross-entropy error:

$$E(\mathbf{w}_1, \dots, \mathbf{w}_K) = -\ln p(\mathbf{T} \mid \Phi, \mathbf{w}_1, \dots, \mathbf{w}_K) = -\sum_{n=1}^N \sum_{k=1}^K t_{nk} \ln y_{nk}.$$

- (e) Write a stochastic gradient algorithm for logistic regression using this objective function. Include indices for time and define the learning rate.

Solution:

For a single sampled data point n_t at iteration t , the per-example cross-entropy is

$$E_{n_t} = -\sum_{k=1}^K t_{n_t k} \ln y_{n_t k}.$$

Since $E = -\ell$, the gradient for class k is

$$\nabla_{\mathbf{w}_k} E_{n_t} = (y_{n_t k} - t_{n_t k}) \phi_{n_t}.$$

With learning rate $\eta^t > 0$, stochastic gradient descent updates all class weights by

$$\mathbf{w}_k^{t+1} = \mathbf{w}_k^t - \eta^t (y_{n_t k} - t_{n_t k}) \phi_{n_t}, \quad k = 1, \dots, K.$$

A simple algorithm is:

- (a) Initialize $\mathbf{w}_1^0, \dots, \mathbf{w}_K^0$ and choose an initial learning rate η^0 .
- (b) For $t = 0, 1, \dots, T-1$:
 - i. Randomly choose an index $n_t \in \{1, \dots, N\}$.
 - ii. Compute $y_{n_t k}$ for all $k = 1, \dots, K$ using the softmax.
 - iii. Update each $\mathbf{w}_k$ using the rule above.
 - iv. Optionally decrease the learning rate.
- (c) Return $\mathbf{w}_1^T, \dots, \mathbf{w}_K^T$.

- (f) In practice, vanilla stochastic gradient descent can be inefficient. Point out a potential weakness of the algorithm and suggest a possible improvement.

Solution:

Possible answers include:

- If the learning rate is too small, convergence can be very slow; if it is too large, the algorithm may diverge.
- Near an optimum, stochastic updates can oscillate because each update uses only one data point. A learning-rate schedule can reduce these oscillations.
- Momentum can help the optimizer move more efficiently through shallow valleys.
- Adaptive optimizers or parameter-specific learning rates can improve practical performance.

- Mini-batches can reduce gradient noise while remaining more efficient than full-batch gradient descent.

Exercise 2: Backpropagation

Consider a two-layer neural network defined as follows:

$$\begin{aligned} \mathbf{z}^{(1)} &= \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}, & \mathbf{a}^{(1)} &= \tanh(\mathbf{z}^{(1)}), \\ \mathbf{z}^{(2)} &= \mathbf{W}^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)}, & \hat{\mathbf{y}} &= \text{softmax}(\mathbf{z}^{(2)}), \end{aligned}$$

where $\mathbf{x} \in \mathbb{R}^D$ is a single input example, $\hat{\mathbf{y}} \in \mathbb{R}^K$ is the predicted output vector, and the hidden layer has H neurons.

- (a) What are the shapes of $\mathbf{W}^{(1)}$, $\mathbf{b}^{(1)}$, $\mathbf{a}^{(1)}$, $\mathbf{W}^{(2)}$, and $\mathbf{b}^{(2)}$?

Solution:

$$\mathbf{W}^{(1)} \in \mathbb{R}^{H \times D}, \quad \mathbf{b}^{(1)} \in \mathbb{R}^{H \times 1}, \quad \mathbf{a}^{(1)} \in \mathbb{R}^{H \times 1}, \quad \mathbf{W}^{(2)} \in \mathbb{R}^{K \times H}, \quad \mathbf{b}^{(2)} \in \mathbb{R}^{K \times 1}.$$

- (b) We feed the neural network with a sample $\mathbf{x}$, obtaining the prediction $\hat{\mathbf{y}}$. We assume we have the one-hot ground-truth label $\mathbf{y}$ for that sample. Write down the cross-entropy loss $L(\mathbf{y}, \hat{\mathbf{y}})$ for that sample.

Solution:

For one sample, the cross-entropy loss is

$$L(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{k=1}^K y_k \log \hat{y}_k.$$

- (c) Compute $\frac{\partial L}{\partial \hat{y}_i}$.

Solution:

Starting from

$$L = - \sum_{j=1}^K y_j \log \hat{y}_j,$$

differentiating with respect to $\hat{y}_i$ gives

$$\frac{\partial L}{\partial \hat{y}_i} = - \frac{y_i}{\hat{y}_i}.$$

- (d) Compute $\frac{\partial L}{\partial z_i^{(2)}}$.

Solution:

By the chain rule,

$$\frac{\partial L}{\partial z_i^{(2)}} = \frac{\partial L}{\partial \hat{\mathbf{y}}} \frac{\partial \hat{\mathbf{y}}}{\partial z_i^{(2)}} = \sum_{j=1}^K \frac{\partial L}{\partial \hat{y}_j} \frac{\partial \hat{y}_j}{\partial z_i^{(2)}}.$$

From the previous part,

$$\frac{\partial L}{\partial \hat{y}_j} = -\frac{y_j}{\hat{y}_j}.$$

The derivative of the softmax (computed in Exercise 1) is

$$\frac{\partial \hat{y}_j}{\partial z_i^{(2)}} = \hat{y}_j(\delta_{ij} - \hat{y}_i),$$

where $\delta_{ij} = 1$ if $i = j$ and 0 otherwise. Plugging this into the chain rule gives

$$\begin{aligned} \frac{\partial L}{\partial z_i^{(2)}} &= \sum_{j=1}^K \left(-\frac{y_j}{\hat{y}_j} \right) \hat{y}_j(\delta_{ij} - \hat{y}_i) \\ &= -\sum_{j=1}^K y_j \delta_{ij} + \hat{y}_i \sum_{j=1}^K y_j. \end{aligned}$$

Since $\mathbf{y}$ is one-hot encoded, $\sum_{j=1}^K y_j = 1$, so

$$\frac{\partial L}{\partial z_i^{(2)}} = \hat{y}_i - y_i.$$

(e) Compute $\frac{\partial L}{\partial W_{ij}^{(2)}}$.

Solution:

By the chain rule,

$$\frac{\partial L}{\partial W_{ij}^{(2)}} = \frac{\partial L}{\partial z_i^{(2)}} \frac{\partial z_i^{(2)}}{\partial W_{ij}^{(2)}}.$$

From the previous part,

$$\frac{\partial L}{\partial z_i^{(2)}} = \hat{y}_i - y_i.$$

Recall that

$$z_i^{(2)} = \sum_{j=1}^H W_{ij}^{(2)} a_j^{(1)} + b_i^{(2)},$$

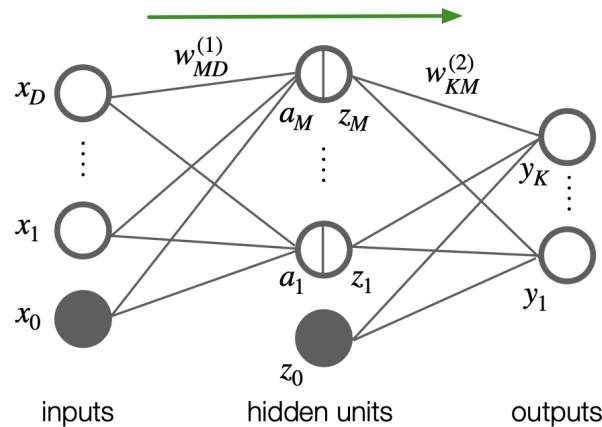
and therefore

$$\frac{\partial z_i^{(2)}}{\partial W_{ij}^{(2)}} = a_j^{(1)}.$$

Hence

$$\frac{\partial L}{\partial W_{ij}^{(2)}} = (\hat{y}_i - y_i) a_j^{(1)}.$$

Consider the following Multilayer Perceptron (MLP) architecture:



The figure shows a two-layer MLP architecture. Note that x_0 and z_0 are bias units, which are always equal to 1.

(a) How many trainable parameters does the model from the figure have?

- (a) $DM + MK$
- (b) $D + K^2$
- (c) $D(D + 1) + K(K + 1)$
- (d) $M(D + 1) + K(M + 1)$
- (e) $D(M + 1) + M(K + 1)$

Solution:

The first layer has M hidden units. Each hidden unit has D input weights and one bias weight, giving

$$M(D + 1)$$

parameters. The second layer has K output units. Each output unit has M incoming weights and one bias weight, giving

$$K(M + 1)$$

parameters. Therefore, the total number of trainable parameters is

$$M(D + 1) + K(M + 1),$$

so the correct answer is (d).

(b) We want to use the architecture sketched in the figure for a classification task, where we receive an input vector $\mathbf{x} \in \mathbb{R}^D$ and want to classify it into one out of K different classes. We have a dataset of N samples with their corresponding ground-truth labels.

- (i) What would we need to modify or add to our current architecture to turn it into a classification model that outputs a class label?
- (ii) What loss function should we optimize?

Note: We are expecting a brief list of necessary layers/functions, indicating the name of the function and what its input would be. You do not need to write any formulas.

Solution:

- (i) Apply a softmax function to the K outputs of the final layer to obtain class probabilities. Then apply an argmax function to either the final-layer output vector or the softmax probability vector to obtain the predicted class label.
 - (ii) Optimize the cross-entropy loss between the softmax output and the corresponding one-hot encoded ground-truth label.
- (c) We want to use the architecture sketched in the figure for a regression task, where we receive an input vector $\mathbf{x} \in \mathbb{R}^D$ and want to predict a single scalar target $t \in \mathbb{R}$. We have a dataset of N samples with their corresponding ground-truth target values.
- (i) What would we need to modify or add to our current architecture to turn it into a regression model that outputs a scalar target?
 - (ii) What loss function should we optimize?

Note: We are expecting a brief list of necessary layers/functions, indicating the name of the function and what its input would be. You do not need to write any formulas.

Solution:

- (i) Add a linear layer with one scalar output after the current final layer, taking the current K outputs as its input. Equivalently, the existing final layer can be replaced by a linear layer with a single output.
- (ii) Optimize the mean squared error between the scalar network output and the corresponding ground-truth target.

Exercise 4: Cross Entropy

Exam question

We are building an animal classifier. The classifier takes a picture of an animal and outputs a probability distribution over the set of possible animals. Assume that we have three classes: cat, dog, and fish. Given a picture of a cat, the model could output a vector such as

$$\mathbf{q} = (0.7, 0.2, 0.1)^\top.$$

This means that the model assigns the picture a 70% probability of being a cat, 20% probability of being a dog, and 10% probability of being a fish.

To measure how well the model is performing, we use the cross-entropy loss. It compares two probability distributions $\mathbf{p}$ and $\mathbf{q}$, where $\mathbf{p}, \mathbf{q} \in \mathbb{R}^n$, and is defined as

$$L(\mathbf{p}, \mathbf{q}) = - \sum_{i=1}^n p_i \log(q_i).$$

In the example above, $n = 3$, the true distribution is

$$\mathbf{p} = (1, 0, 0)^\top,$$

and the predicted distribution is

$$\mathbf{q} = (0.7, 0.2, 0.1)^\top.$$

We assume that $\mathbf{p}$ is always one-hot encoded, meaning that all entries are zero except for one entry equal to one. In this exercise, use the convention $0 \log(0) = 0$, which is natural because $\lim_{x \rightarrow 0^+} x \log x = 0$.

(a) What is the value of $L(\mathbf{p}, \mathbf{q})$ for this example?

Solution:

Since $\mathbf{p} = (1, 0, 0)^\top$ and $\mathbf{q} = (0.7, 0.2, 0.1)^\top$,

$$\begin{aligned} L(\mathbf{p}, \mathbf{q}) &= - \sum_{i=1}^3 p_i \log(q_i) \\ &= -(1 \cdot \log(0.7) + 0 \cdot \log(0.2) + 0 \cdot \log(0.1)) \\ &= -\log(0.7). \end{aligned}$$

(b) Prove that $L(\mathbf{p}, \mathbf{q}) \geq 0$ for all probability distributions $\mathbf{p}, \mathbf{q} \in \mathbb{R}^n$.

Solution:

Since $\mathbf{q}$ is a probability distribution, each entry satisfies $0 \leq q_i \leq 1$. Therefore,

$$\log(q_i) \leq 0$$

for all i for which $q_i > 0$. Also, since $\mathbf{p}$ is a probability distribution, $p_i \geq 0$. Thus

$$p_i \log(q_i) \leq 0.$$

Therefore

$$L(\mathbf{p}, \mathbf{q}) = - \sum_{i=1}^n p_i \log(q_i) \geq 0.$$

(c) When does $L(\mathbf{p}, \mathbf{q}) = 0$? Remember that $\mathbf{p}$ is one-hot encoded.

Solution:

Suppose the correct class is c , so $p_c = 1$ and $p_i = 0$ for all $i \neq c$. Then

$$L(\mathbf{p}, \mathbf{q}) = -\log(q_c).$$

This is zero exactly when

$$\log(q_c) = 0,$$

which happens when $q_c = 1$. Since $\mathbf{q}$ is a probability distribution, this also implies $q_i = 0$ for all $i \neq c$. Therefore, the loss is zero exactly when the prediction $\mathbf{q}$ equals the one-hot ground truth $\mathbf{p}$.

(d) When is $L(\mathbf{p}, \mathbf{q})$ maximized? Remember that $\mathbf{p}$ is one-hot encoded.

Solution:

Again, let c be the correct class, so $p_c = 1$. Then

$$L(\mathbf{p}, \mathbf{q}) = -\log(q_c).$$

The loss becomes larger as q_c becomes smaller. In the limiting case where the model assigns probability zero to the correct class, $q_c = 0$, we get

$$L(\mathbf{p}, \mathbf{q}) = -\log(0) = +\infty.$$

Thus the cross-entropy loss is unbounded above; it is maximized in the limit where the predicted probability of the true class goes to zero.